

DATA SCIENCE PROJECT

QUESTION

A government database stores the following identification numbers:

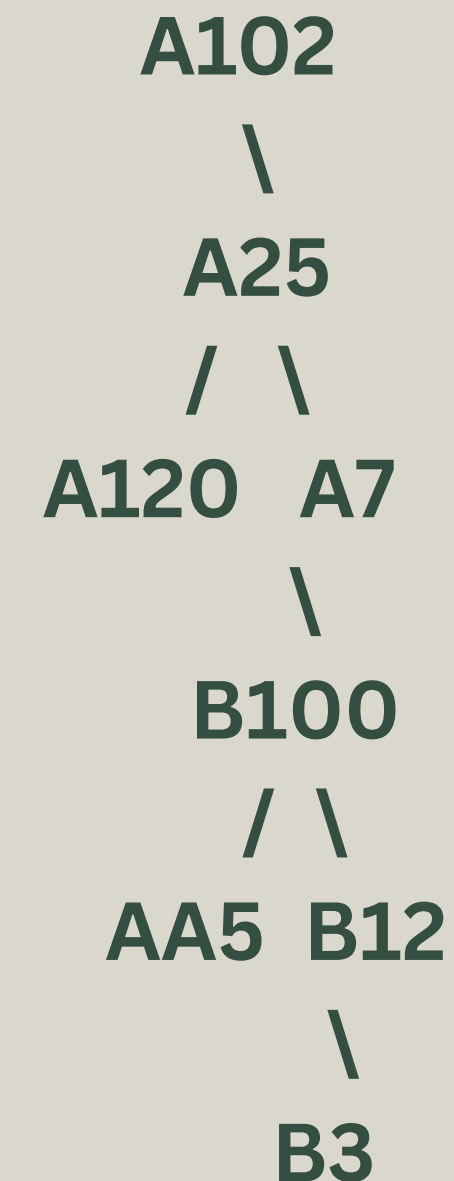
A102, A25, A7, B100, B12, A120, B3, A45

- a) Implement a BST to organise the identification numbers and display its inorder traversal. Analyse the resulting tree structure.
- b) Compare the performance of BST Search and Linear Search for selected identification numbers. Record the number of comparisons.
- c) Analyse how varying key lengths and insertion order affect the BST height and search performance. Compare the observed performance with the theoretical best-case and worst-case complexities.

Suggest a suitable approach for maintaining efficient searches as the database grows.

GRAPH BINARY SEARCH TREE

- An inorder traversal (Left, Root, Right) of a valid BST always lists the keys in sorted, ascending order:
(\text{Inorder:} \{ \mathbf{A102, A120, A25, A7, AA5, B100, B12, B3} \}) If you need help with the next parts of your assignment, let me know if you would like to: Solve part b by counting the exact number of comparisons for a linear search versus a BST search. Analyze part c regarding how key length and insertion order skew the tree structure. e bit of body text



PROGRAM

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  #include<string.h>
4  #define N 8
5  typedef struct Node{
6      char id[20];
7      struct Node *left;
8      struct Node *right;
9  } Node;
10 /*function declaration*/
11 int bstSearch(Node *root,char key[],int*comparisons);
12 int linearSearch(char ids[][20],char key[],int*comparisons);
13
14 /*create a new BST node*/
15 Node* createNode(char id[]){
16     Node *newNode=(Node*)malloc(sizeof(Node));
17     strcpy(newNode->id,id);
18     newNode->left = NULL;
19     newNode->right = NULL;
20
21     return newNode;
22 }
23 /*insert an ID into BST*/
24 Node* insert(Node *root,char id[])
25 {
26     if(root ==NULL)
27         return createNode(id);
28     if(strcmp(id,root->id)<0)
29         root->left=insert(root->left,id);
30     else if (strcmp(id,root->id)>0)
31         root->right=insert(root->right,id);
32     return root;
33 }
```

```

34  /*Inorder traversal*/
35  void inorder(Node *root)
36  {
37      if(root!=NULL)
38      {
39          inorder(root->left);
40          printf("%s",root->id);
41          inorder(root->right);
42      }
43  }
44  /*find height of BST*/
45  int height(Node *root)
46  {
47      if(root == NULL)
48          return 0;
49      int leftHeight=height(root->left);
50      int rightHeight=height(root->right);
51      return (leftHeight>rightHeight ? leftHeight:rightHeight)+1;
52  }
53
54  /*BST search with comparison count */
55  int bstSearch(Node *root,char key[],int *comparisons)
56  {
57      while(root !=NULL)
58      {
59          (*comparisons)++;
60          int result = strcmp(key,root->id);
61          if(result==0)
62              return 1;
63          if(result<0)
64              root=root->left;
65          else
66              root=root->right;

```

```

66         root=root->right;
67     }
68     return 0;
69 }
70 int linearSearch(char ids[][20],char key[],int *comparisons)
71 {
72     for(int i=0;i<N;i++)
73     {
74         (*comparisons)++;
75         if (strcmp(ids[i],key)==0)
76             return 1;
77     }
78     return 0;
79 }
80 /*build a BST for a particular insertion order*/
81 Node* buildTree(char ids[][20])
82 {
83     Node *root = NULL;
84     for (int i=0;i < N;i++)
85         root = insert(root,ids[i]);
86     return root;
87 }
88 /*test searching performance*/
89 void compareSearch(Node *root,char ids[][20],char key[])
90 {
91     int bstComp = 0;
92     int linearComp = 0;
93     bstSearch(root,key,&bstComp);
94     linearSearch(ids,key,&linearComp);
95     printf("%s-%s %d-%d\n",key,bstComp,linearComp);
96 }
97 int main()
98 {

```

```

97  int main()
98  [= {
99      char ids[N][20] = {" A102", " A25", " A7", " B100", " B12", " A120", " B3", " A45"};
100
101  char sorted[N][20] = {" A102", " A120", " A7", " B100", " B12", " B3"};
102  char reverse[N][20] = {" B3", " B12", " B100", " A7", " A25", " A102"};
103  /*original insertion order*/
104  Node *root = buildTree(ids);
105  printf("-----GOVERNMENT DATABASE BST-----\n\n");
106  printf("Identification Numbers:\n");
107  for (int i =0;i<N;i++)
108      printf("%s",ids[i]);
109  printf("\n\n Inorder Traversal:\n ");
110  inorder(root);
111  printf("\n\n BST Height =%d\n",height(root));
112  printf("\n-----SEARCH PERFORMANCE-----\n");
113  printf("%-8s %-15s %-15s\n","ID","BST","Linear");
114  printf("-----\n");
115  compareSearch(root,ids," A45");
116  compareSearch(root,ids," B3");
117  compareSearch(root,ids," A120");
118  /*analyse insertion order*/
119  Node *sortedTree=buildTree(sorted);
120  Node *reverseTree=buildTree(reverse);
121  printf("\n-----INSERTION ORDER ANALYSIS-----\n");
122  printf("Given order :Height=%d\n",height(root));
123  printf("Sorted order : Height=%d\n",height(sortedTree));
124  printf("Reverse order : Height =%d\n",height(reverseTree));
125  printf("\n-----COMPLEXITY-----\n");
126  printf("BST Search | Best Case: O(n)\n");
127  printf("Linear Search : O(n)\n");
128  return 0;
129  }

```

OUTPUT

"C:\Users\ADMIN\Documents X

-----GOVERNMENT DATABASE BST-----

Identification Numbers:

A102 A25 A7 B100 B12 A120 B3 A45

Inorder Traversal:

A102 A120 A25 A45 A7 B100 B12 B3

BST Height =6

-----SEARCH PERFORMANCE-----

ID	BST	Linear
----	-----	--------

A45	4	8
B3	6	7
A120	3	6

-----INSERTION ORDER ANALYSIS-----

Given order :Height=6

Sorted order : Height=6

Reverse order : Height =7

-----COMPLEXITY-----

BST Search - Best Case: $O(n)$

Linear Search : $O(n)$

Process returned 0 (0x0) execution time : 20.975 s

Press any key to continue.

Complexity output

A102 A25 A7 B100 B12 A120 B3 A45

Inorder Traversal:

A102 A120 A25 A45 A7 B100 B12 B3

BST Height =6

-----SEARCH PERFORMANCE-----

ID	BST	Linear
----	-----	--------

A45	4	8
B3	6	7
A120	3	6

-----INSERTION ORDER ANALYSIS-----

Given order :Height=6

Sorted order : Height=8

Reverse order : Height =8

-----COMPLEXITY-----

BST Search - Best Case: $O(n)$

BST Search - Average: $O(\log n)$

BST Search - Worst Case : $O(n)$

Linear Search - Best Case: $O(1)$

Linear Search - Average: $O(n)$

Linear Search - Worst Case: $O(n)$

Process returned 0 (0x0) execution time : 1.567 s

Press any key to continue.

Conclusion

> The Binary Search Tree (BST) was successfully implemented to store and search government identification numbers. The experiment showed that BST search can require fewer comparisons than linear search when the tree structure is suitable. It also demonstrated that the insertion order affects the height of the BST and therefore its search performance. A balanced or reasonably shaped BST provides better search efficiency, while a skewed BST can have $O(n)$ search time. Thus, BST is a useful data structure for managing and searching a growing collection of identification records when its height is kept under control.

Thank you